



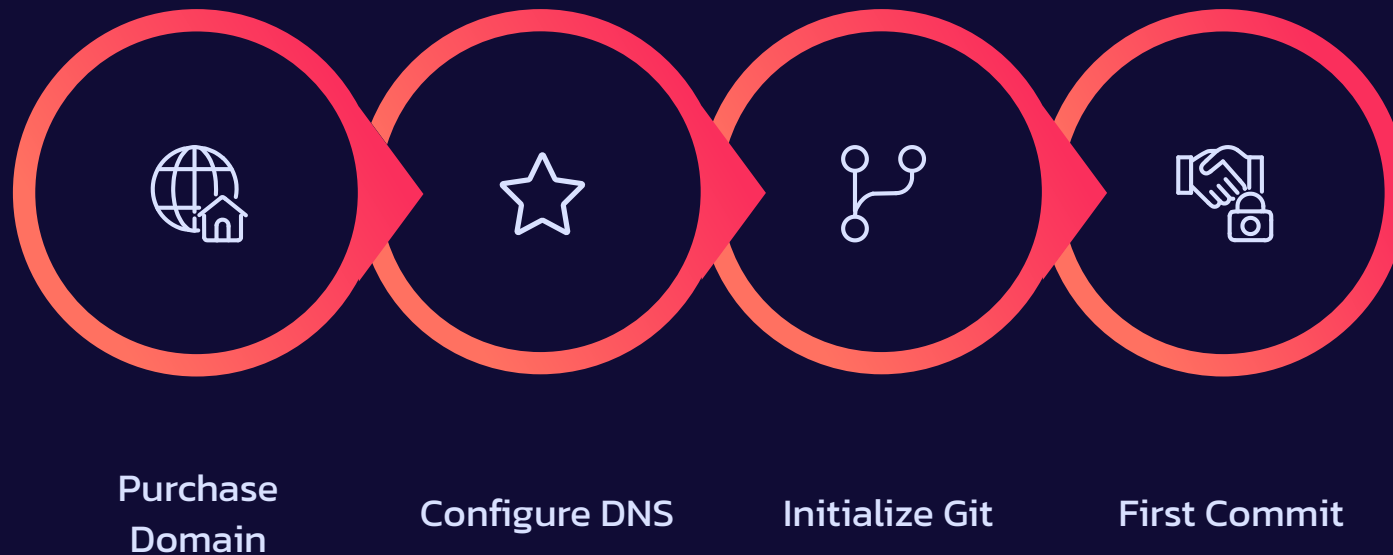
CS & Web Fundamentals

A hands-on journey from domain setup and Git workflows to low-level C programming, Linux environments, and system architecture — everything you need to build a rock-solid computer science foundation.

CHAPTER 1

From Domain to First Commit

Every web project starts the same way: claim your space on the internet, then lock in your code with version control.



Run `git init`, create your `index.html`, then stage and commit with `git add index.html` and `git commit -m "Initial commit"` — your work is now tracked.

Algorithmic Principles & Design Practices



User Experience

Design intuitive prompts, handle errors gracefully, and produce readable outputs.



Meaningful Names

Use self-documenting identifiers like `userAge`.



Correctness

Verify against formal specs using edge cases, unit tests, and dry-runs.



Coding Style

Apply consistent linting, spacing, and indentation.



Multiple Solutions

Weigh alternative data structures and algorithm paradigms for trade-offs.



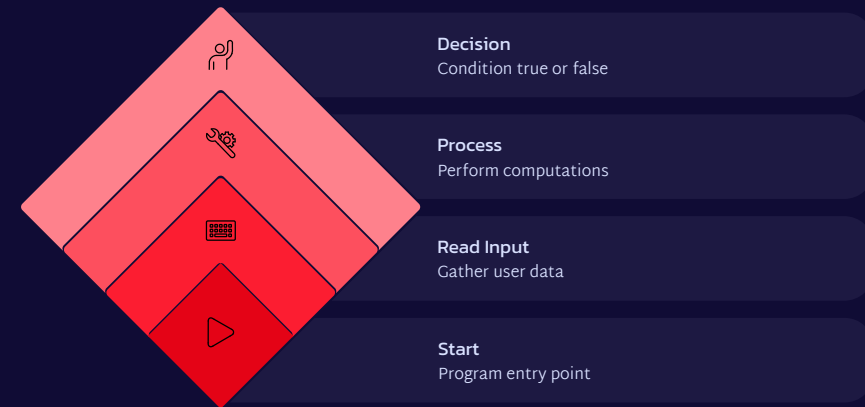
DRY Principle

Keep code reusable and avoid duplication.

Optimization still matters: analyze time $O(N)$ vs $O(1)$ and space complexity to improve continuously.

Flowcharts: Visualize Before You Code

Before writing a single line of C, map your logic with a flowchart. Standard symbols keep execution clear and catch design flaws early.



Start / Stop

Mark program entry and exit boundaries.

Read / Print

Control all input and output operations.

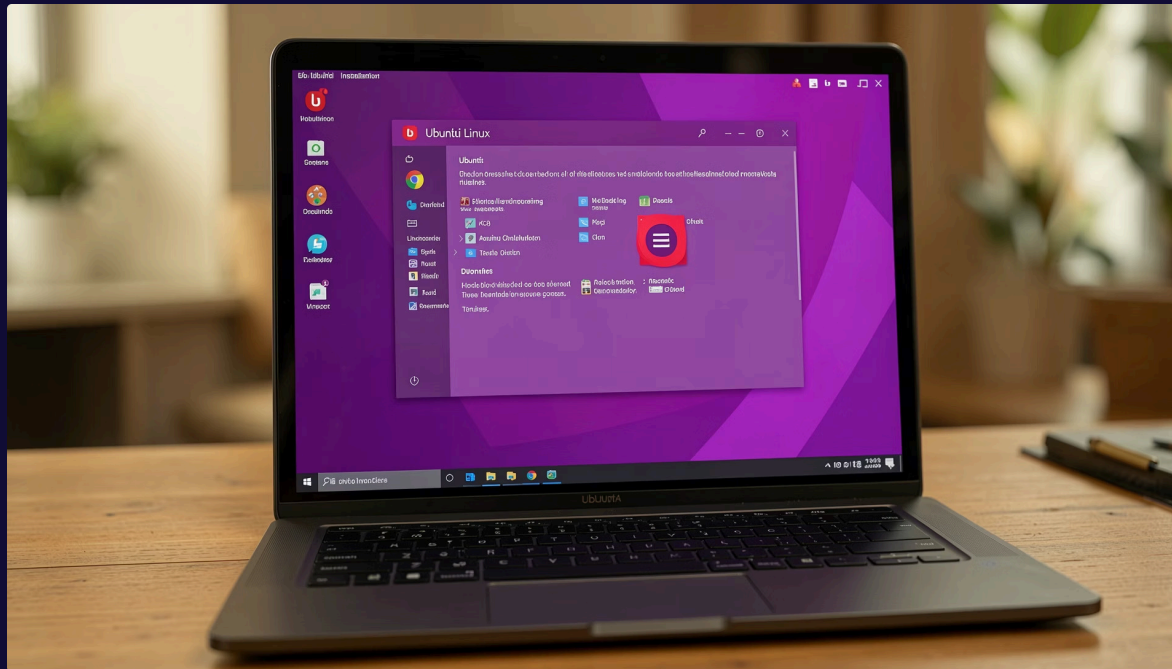
Process

Perform calculations and data transformations.

Condition

Branch execution based on true/false decisions.

Ubuntu Linux: Install & Navigate



Installation Steps

1. **Download ISO:** Grab the latest Ubuntu LTS from Canonical.
2. **Create Bootable USB:** Flash the ISO with Rufus or BalenaEtcher.
3. **Install:** Boot from USB, partition disk as ext4, and set up your user.

Essential Commands

- `ls / pwd / cd` — Navigate the file system
- `mkdir / rm / mv / cp` — Manage files
- `chmod` — Change file permissions
- `man` — Read command documentation

Writing Your First C Program

Program Structure

Every C program starts with `#include` preprocessor directives, executes inside `int main()`, and ends with `return 0;` to signal success.

Example Code

```
#include <stdio.h>

int main() {
    int userAge = 21;
    printf("Welcome to CS Fundamentals!\n");
    printf("Age: %d\n", userAge);
    return 0;
}
```

Preprocessor

`#include <stdio.h>` loads standard I/O.

Main Function

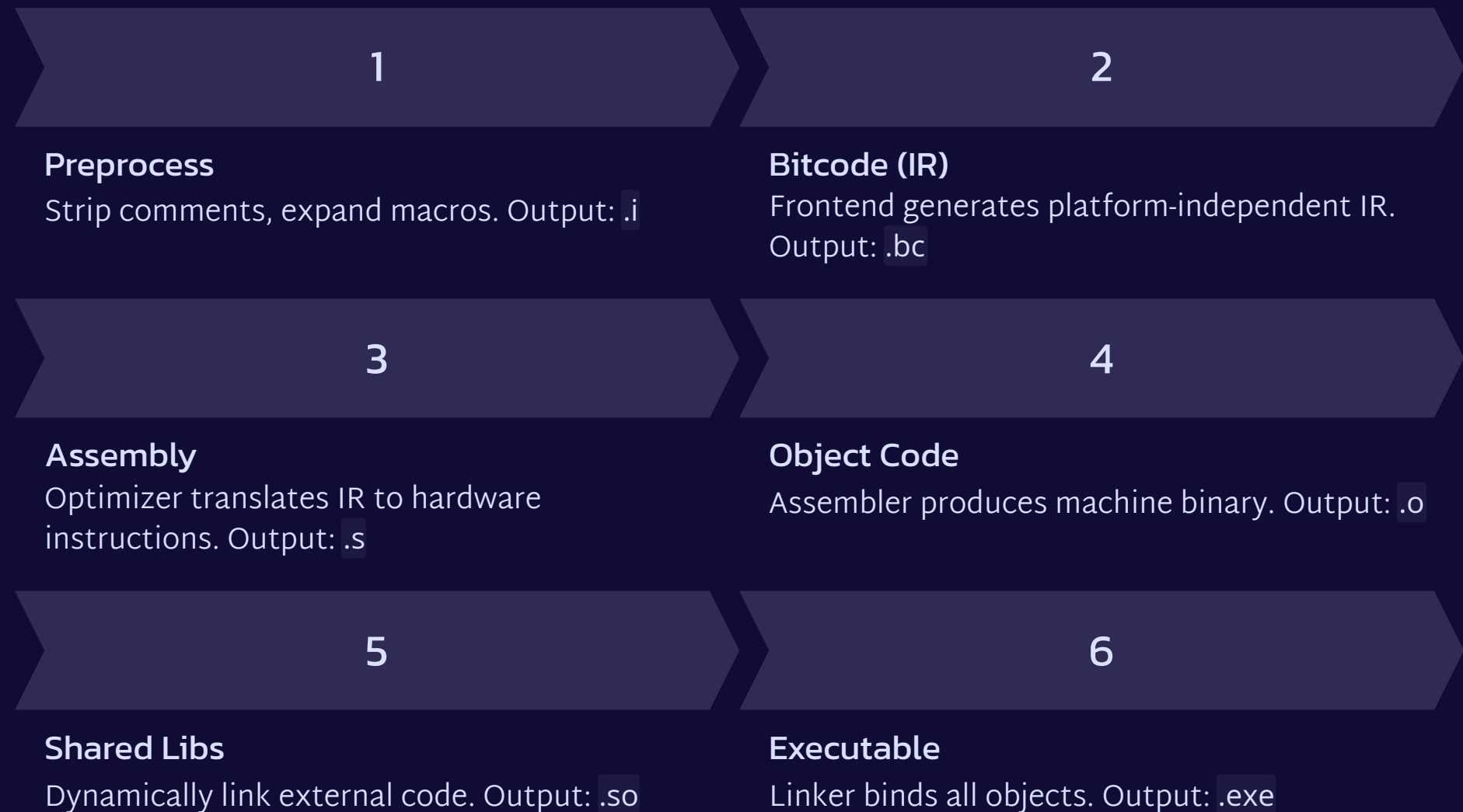
Entry point — the OS calls `main()` first.

Return Value

`return 0;` signals clean exit to the OS.

The C Compilation Pipeline

When you run `gcc`, your source code passes through six distinct stages before becoming a runnable executable.



Git: Snapshots, SSH & Trust

Local Git Workflow

1 Status

`git status` shows modified and untracked files.

2 Stage

`git add` moves changes to the staging area.

3 Commit

`git commit -m "Msg"` saves a permanent snapshot.

Building Trust with Cryptography

SSH Keys

Authenticate with remote servers securely — no passwords needed.

GPG Signing

Run `git commit -S` to cryptographically prove authorship.

Verified Badge

Platforms display a "Verified" badge for signed commits, establishing code provenance.

Hardware Architecture & Number Systems

The System Bus Model

All hardware components communicate through a shared **system bus**.



CPU

ALU + Control Unit execute instructions.



Memory & Disk

RAM for fast access; disk for persistent storage.



I/O Devices

Keyboards, monitors, and printers interface with the user.

Number Systems

Decimal	Binary	Octal	Hex
0	0000	0	0
5	0101	5	5
10	1010	12	A
15	1111	17	F
16	10000	20	10

Computers process binary natively. Hexadecimal concisely encodes 4 bits (a nibble) per character — essential for memory addresses and debugging.

CHAPTER 9

Debugging with LLDB

Compile with `gcc -g` to embed debug symbols, then launch LLDB to step through execution and inspect memory.

01

Start & Break

Run `lldb ./a.out`, then set a breakpoint at `main` to pause at startup.

02

Step & Inspect

Use `n` to step over, `s` to step into functions. Print values with `p var_name`.

03

Stack Trace

On a segfault, run `bt` (backtrace) to see the exact call stack leading to the crash.



Key Takeaway: Mastering LLDB turns crashes into learning opportunities — every segfault teaches you how memory works.

